

Range Sum Query (Immutable) / Prefix Sum

1. What is an Immutable Range Sum Query?

You are given an array of numbers.

You will be asked multiple times to find the sum of elements between a `LEFT` index and a `RIGHT` index.

"Immutable" means the array is static — the numbers inside the array never change.

2. Why is the Naive Way Bad?

A simple way is to use a `for` loop from `LEFT` to `RIGHT` every time a query is asked.

But if the array is huge, and you get thousands of queries, looping every single time is too slow!

Time for each query: $O(N)$

3. The Main Idea: Prefix Sum

Instead of calculating from scratch every time, we pre-calculate the running totals once.

A Prefix Sum array stores the sum of all elements from the start `0` up to index `i`.

```
pref[i] = sum of elements from index 0 to i
```

4. Most Important Formula

Range Sum Formula:

```
sum = pref[R] - pref[L - 1]
```

Why?

Think of a long piece of string.

You want the length of the middle part (from L to R).

TOTAL STRING up to R = `pref[R]`

STRING BEFORE L = `pref[L - 1]`

To get the middle part, take the total string up to R, and cut off the string before L:

MIDDLE = TOTAL - BEFORE

Therefore:

```
sum = pref[R] - pref[L - 1]
```

(Note: If $L == 0$, there is nothing to cut off, so the sum is just $pref[R]$)

5. Step-by-Step Example

Consider:

```
nums = [2, 4, 3, 5, 1, 6]
```

Step A: Build the Prefix Sum array

Keep a running total:

Index 0: 2

Index 1: $2 + 4 = 6$

Index 2: $6 + 3 = 9$

Index 3: $9 + 5 = 14$

Index 4: $14 + 1 = 15$

Index 5: $15 + 6 = 21$

So, $pref = [2, 6, 9, 14, 15, 21]$

Step B: Answer a Query

Find sum between $LEFT = 1$ and $RIGHT = 3$.

Using our formula:

```
sum = pref[RIGHT] - pref[LEFT - 1]
```

```
sum = pref[3] - pref[0]
```

Look at our `pref` array:

```
pref[3] = 14
```

```
pref[0] = 2
```

```
sum = 14 - 2 = 12
```

Check manually: $nums[1] + nums[2] + nums[3] = 4 + 3 + 5 = 12$. It matches!

6. Visual Understanding

```
[ 2 | 4 3 5 | 1 6 ]
```

```
\___/ \_____/
```

BEFORE TARGET

(L-1) (L to R)

pref[R] covers everything up to index 3 (2 + 4 + 3 + 5)

pref[L-1] covers everything up to index 0 (2)

Subtracting them leaves only the TARGET (4 + 3 + 5).

7. Python Code (Using standard 0-indexing)

```
class NumArray:
    def __init__(self, nums: list[int]):
        self.pref = []
        current_sum = 0
        for num in nums:
            current_sum += num
            self.pref.append(current_sum)

    def sumRange(self, left: int, right: int) -> int:
        if left == 0:
            return self.pref[right]
        else:
            return self.pref[right] - self.pref[left - 1]
```

8. Quick Formula to Remember

The whole concept can be reduced to this:

Pre-computation:

pref[i] = pref[i-1] + nums[i]

Query:

Query(L, R) = pref[R] - pref[L - 1]

9. Complexity

Time:

- Building the array: O(N) once.
- Answering a query: O(1) instant!

Space: $O(N)$

We use an extra array of the same size to store the prefix sums.

10. Final Mental Model

Whenever you see "multiple sum queries on a static array", think:

↓

[PREFIX BEFORE L] [TARGET FROM L TO R]

TOTAL UP TO R MINUS PREFIX BEFORE L == TARGET

And use:

```
sum = pref[R] - pref[L - 1]
```